arXiv:2603.12228v1 [cs.LG] 12 Mar 2026

Neural Thickets:
Diverse Task Experts Are Dense Around Pretrained Weights

Yulu Gan, Phillip Isola
MIT CSAIL
{yulu_gan, phillipi}@mit.edu

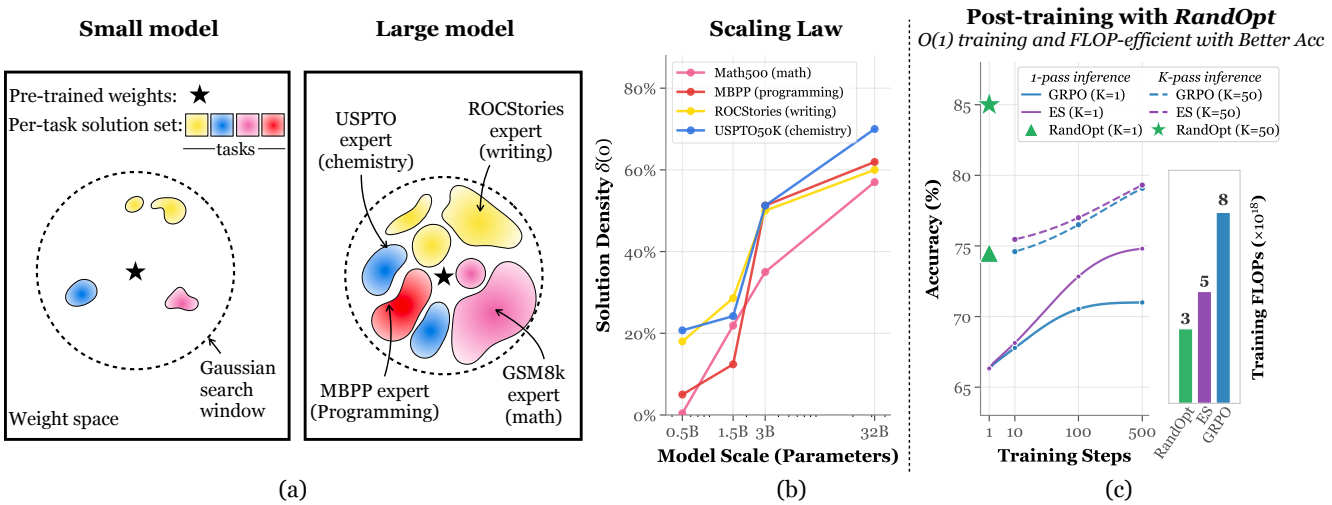

Figure 1: (a) Schematic of the main effects we observe (see Fig 2 for a version with real data). Left: Small models live in a *needle in a haystack* regime, where good solutions to downstream tasks occupy a tiny fraction of the surrounding weights. In this regime, it is important to have a smart search algorithm, such as gradient descent or other forms of iterative optimization. Right: Large models are surrounded by a veritable *thicket* of task-specific solutions. In this regime, random sampling is sufficient to quickly land on promising adaptations, which can then be ensembled to yield strong behavior, an approach we call *RandOpt*. (b) Solution density – i.e. density of task-improving weights in a Gaussian neighborhood of the pretrained weights – scales with model size. (c) RandOpt is $\mathcal{O}(1)$ in training steps, FLOP-efficient, and competitive in converged accuracy with GRPO and ES. Results are shown on the Countdown task with `Olmo-3-7B-Instruct`; RandOpt uses 5000 random weight guesses and ensembles the top K; K-pass baselines use Test-time Majority Vote (TT-MV). More results are shown in Fig. 6 and Table 4.

Abstract

Pretraining produces a learned parameter vector that is typically treated as a starting point for further iterative adaptation. In this work, we instead view the outcome of pretraining as a distribution over parameter vectors, whose support already contains task-specific experts. We show that in small models such expert solutions occupy a negligible fraction of the volume of this distribution, making their discovery reliant on structured optimization methods such as gradient descent. In contrast, in large, well-pretrained models the density of task-experts increases dramatically, so that diverse, task-improving specialists populate a substantial fraction of the neighborhood around the pretrained weights. Motivated by this perspective, we explore a simple, fully parallel post-training method that samples N parameter perturbations at random, selects the top K, and ensembles predictions via majority vote. Despite its simplicity, this approach is competitive with standard post-training methods such as PPO, GRPO, and ES for contemporary large-scale models.

Project page: https://thickets.mit.edu Code: https://github.com/sunrainyg/RandOpt

1 Introduction

"[Random guessing] cannot be viewed as a reasonable learning algorithm..."
— Schmidhuber, Hochreiter, Bengio, 2001

One of the first algorithms we learn in elementary school is "guess and check." In its simplest form the procedure is almost trivial: given an equation with unknowns, guess values of the unknowns and check whether they satisfy the equation. Guess again, completely at random, until it works. The same approach can be applied to machine learning, but it has long been assumed to be hopeless, as the quote above implies (Schmidhuber et al., 2001). Consider, for example, the chance of randomly guessing, from scratch, a billion-dimensional parameter vector that behaves like ChatGPT. The probability must be astronomically small.

This paper finds that after pretraining, the story changes. With a reasonable number of random guesses, one can sample parameter perturbations that substantially improve pretrained large language models (LLMs) across a broad set of tasks. How is this possible? For random guessing to work, good solutions must be dense under the distribution being sampled from. Schmidhuber, Hochreiter, and Bengio made precisely this point in their 2001 paper: random guessing *did* solve some benchmark problems of that era. However, they interpreted this as a failure of the benchmarks to assess difficult skills. We instead show that the same phenomenon occurs in a contemporary setting of practical interest: post-training LLMs on tasks such as reasoning, programming, and more.

How does the loss landscape change after pretraining, in order that random guessing begins to work? We study two effects. First we measure the *density* of task-improving solutions in a Gaussian neighborhood around the pretrained weights. We find that this density increases with pretraining scale. Untrained models have a tiny density of solutions near their initial weights; they live in a *needle in a haystack* regime, where finding the solution requires structured multi-step search, such as gradient descent. Conversely, large pretrained models transition into a regime of high density, replete with task-experts near the pretrained weights. We term this the *thicket regime*.

Next, we study solution *diversity* in the Gaussian neighborhood. It turns out that the different sampled parameter vectors are not uniform improvements. They are specialists rather than generalists, where the perturbations that most improve performance on one task hurt performance on other tasks. We visualize these two effects in Figure 1.

Motivated by these findings, we explore a fully parallel post-training algorithm that exploits the density and diversity of pretrained neighborhoods. The method is a form of random guessing (which works due to the neighborhood density) followed by ensembling (which exploits the neighborhood diversity). Given an initial weight vector, N perturbations of that vector are created. Each perturbation is evaluated on the post-training data. The top K perturbations are selected and their predictions are ensembled via majority voting. We call this algorithm RandOpt.

RandOpt achieves accuracy competitive with PPO, GRPO, and ES under the same post-training flops. On the wall-clock it trains in $\mathcal{O}(1)$ time compared to $\mathcal{O}(T)$ for the baselines that require T sequential update steps. Its inference-time cost is K times higher due to ensembling. For some tasks, $K = 1$ already achieves decent results. For other tasks, only larger K is competitive with the baselines, but in Section 7, we show a proof of concept that this cost can be reduced via distillation. However, our goal is not to promote RandOpt as superior to alternative methods. Rather, we use it as a probe: its success suggests that *post-training becomes easy once you have a strong pretrained representation* – i.e., once you enter the thicket regime. In that regime, it doesn't matter much which method you use – gradient-based search, evolutionary algorithms, and brute-force parallel selection all will do.

Main Findings

1. In large models, the neighborhood around pretrained weights is dense with task-improving solutions (Fig 2).
2. The density exhibits a scaling law, with higher density for larger, more performant models (Fig 3).
3. The local neighborhood is also diverse: individual perturbations tend to improve performance on particular tasks while degrading others (Fig 4). Diversity also scales with model size (Fig 3).
4. For current models, the density is high enough that random guessing is effective for post-training (Fig 6).
5. Ensembling over multiple guessed solutions further improves performance, often substantially (Fig 11).

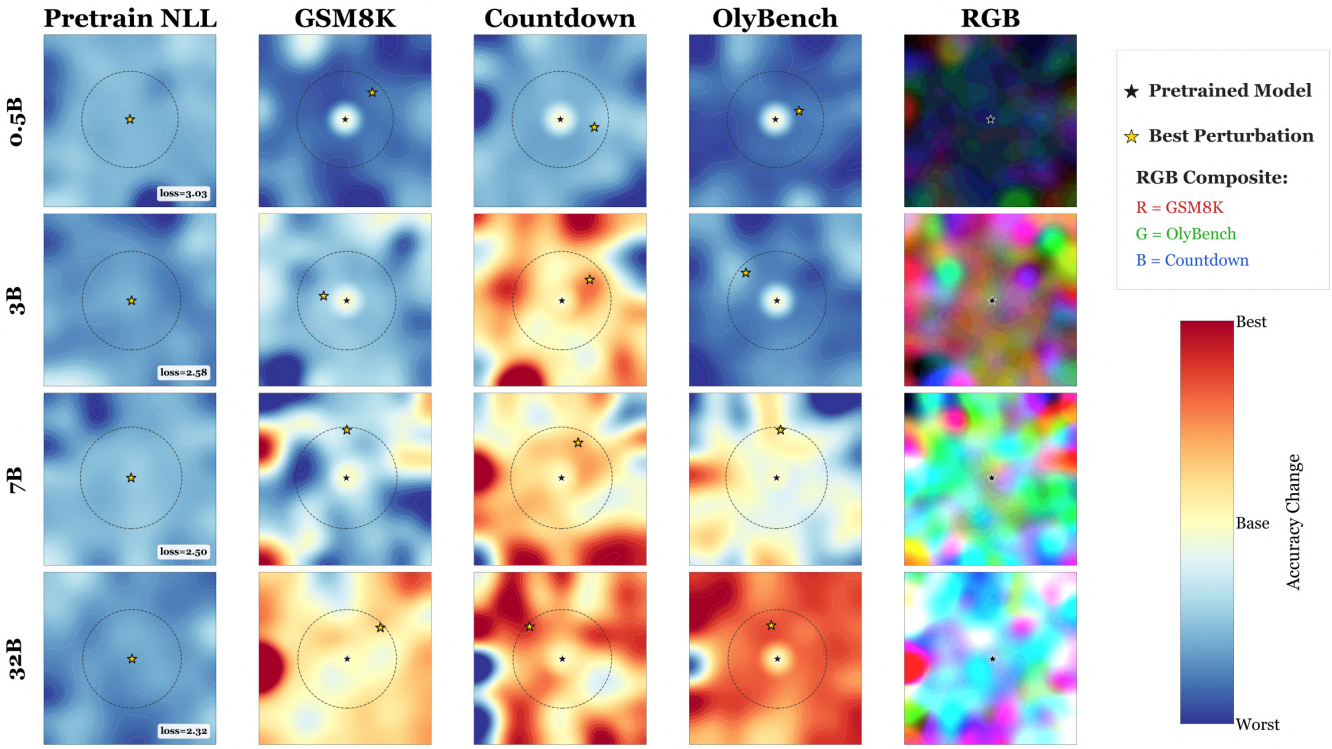

Figure 2: Accuracy landscapes in weight space across model scales and reasoning tasks. We perturb the pretrained Qwen2.5 models (from 0.5B to 32B) with 1000 random weight perturbations and project the perturbed models into 2D using random projection. Colors show relative accuracy change $(\text{acc} - \text{base})/\text{base} \times 100$ (blue: degraded, white: equivalent, red: improved.) Dashed circles indicate the mean perturbation distance and stars mark the best-performing perturbations. Larger models have warmer landscapes, with more high-performing neighborhoods. The last column shows an RGB visualization where GSM8K, Olympiad and Countdown accuracies are mapped to R,G,B channels; richer colors indicate more task experts.

2 Structure of the Multi-task Loss Landscape Around Pretrained Weights

In this section, we measure the density and diversity of task-improving solutions in the vicinity of pretrained weights, across models of different scale and a variety of tasks.

2.1 Solution Density: What Proportion of Local Perturbations Improve Task Performance?

Figure 2 visualizes the performance of Gaussian-perturbed models across scales from 0.5B to 32B parameters, and on three reasoning tasks. A topographic shift is evident: small models reside on local maxima of the accuracy landscape, whereas larger models inhabit an accuracy "valley," with many peaks of higher accuracy nearby (red regions). This indicates that scaling may fundamentally reshape the loss landscape. Given this change, what is the probability of finding good solutions? We define the solution density as:

Definition 2.1 (Solution Density)**.** Let $s : \mathbb{R}^d \to \mathbb{R}$ be the performance evaluation metric for model parameters $\boldsymbol{\theta} \in \mathbb{R}^d$. We define the *Solution Density* $\delta(m)$ as the probability that a random perturbation $\boldsymbol{\epsilon}$ improves the base model's score by a margin m:

$$\delta(m) = \mathbb{P}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})} \left[s(\boldsymbol{\theta} + \boldsymbol{\epsilon}) \geq s(\boldsymbol{\theta}) + m \right] \tag{1}$$

where σ scales the local Gaussian neighborhood (in the experiments in this section, we use $\sigma = 0.005$). Intuitively, $\delta(m)$ measures the "hit rate" of random guessing: it quantifies the proportion of the explored parameter space that yields a performance gain of at least m.

In Figure 3, we measure $\delta(m)$, for several values of m, as a function of model scale. Solution density increases monotonically with model size, and this trend holds over multiple values of m (e.g., the density of perturbations that increase performance by +5% accuracy increases monotonically as a function of model scale). These results